CS 301
High-Performance Computing

Lab Report-03
Scattered Data Interpolation
Performance Analysis

Atik Vohra (202301447)
Soham Mevada (202301484)

Contents

1 Introduction **3**

2 Hardware Details **3**
2.1 LAB207 PCs 3
2.2 HPC Cluster 3

3 Methodology **4**

4 Scattered Data Interpolation **5**
4.1 Problem Size vs Time 5
4.1.1 Local System 5
4.1.2 HPC Cluster 5

5 Answers to Analysis and Questions **6**
5.1 1. Theoretical Time Complexity 6
5.2 2. Arithmetic Operations per Particle 6
5.3 3. Memory Access and Cache Behavior Analysis 7
5.4 4. Possible Optimizations Under Better Hardware 8
5.5 5. Execution Time Comparison (Lab PC vs HPC Cluster) 8
5.6 6. Implementation Explanation (Conceptual Description) 9
5.7 Pictorial Explanation of Interpolation Method 9

6 Conclusion **10**

1 Introduction

This assignment implements a serial 2D interpolation algorithm that maps scattered data points onto a structured M×M grid using linear (bilinear) interpolation. Each scattered point distributes its value to its four surrounding grid points based on relative distances.

The focus of this lab is to ensure correctness of the implementation, analyze computational complexity and memory access behavior, and evaluate performance across different problem sizes and hardware systems.

2 Hardware Details

2.1 LAB207 PCs

- Architecture: x86_64
- CPU(s): 12
- Threads per core: 2
- Cores per socket: 6
- Processor: Intel Core i5-12500
- L1 Cache: 288K
- L2 Cache: 7.5M
- L3 Cache: 18M

2.2 HPC Cluster

- Architecture: x86_64
- CPU(s): 16
- Threads per core: 1
- Cores per socket: 8
- Sockets: 2
- Processor: Intel Xeon E5-2640 v3
- L1 Cache: 32K
- L2 Cache: 256K
- L3 Cache: 20480K

3 Methodology

The interpolation routine implements a two–dimensional bilinear interpolation (also known as the Cloud-In-Cell method in particle–mesh schemes) on a structured Cartesian grid of size $N_X \times N_Y$ with grid spacing Δx and Δy.

1. Cell Identification For a particle located at physical position (x, y) in a normalized domain $[0, 1] \times [0, 1]$, the containing cell is determined as

$$i = \left\lfloor \frac{x}{\Delta x} \right\rfloor, \qquad j = \left\lfloor \frac{y}{\Delta y} \right\rfloor.$$

This identifies the lower-left grid node (i, j) of the cell containing the particle.
Boundary clamping ensures

$$0 \le i \le N_X - 1, \qquad 0 \le j \le N_Y - 1,$$

preventing out-of-bounds memory access.

2. Local Coordinates Within the Cell The particle's local coordinates inside the cell are

$$\ell_x = x - i\Delta x, \qquad \ell_y = y - j\Delta y,$$

where

$$0 \le \ell_x \le \Delta x, \qquad 0 \le \ell_y \le \Delta y.$$

The complementary distances to the opposite cell faces are

$$w_x^- = \Delta x - \ell_x, \qquad w_y^- = \Delta y - \ell_y.$$

3. Bilinear Weights The particle contributes to the four surrounding grid nodes:

$$(i, j), \quad (i + 1, j), \quad (i, j + 1), \quad (i + 1, j + 1).$$

The bilinear weights are constructed as separable products:

$$\begin{aligned} w_{00} &= (\Delta x - \ell_x)(\Delta y - \ell_y), \\ w_{10} &= \ell_x(\Delta y - \ell_y), \\ w_{01} &= (\Delta x - \ell_x)\ell_y, \\ w_{11} &= \ell_x \ell_y. \end{aligned}$$

4. Grid Accumulation Using row-major indexing,

$$\text{index}(i, j) = j \cdot N_X + i,$$

the weighted contributions are accumulated to the four neighboring nodes:

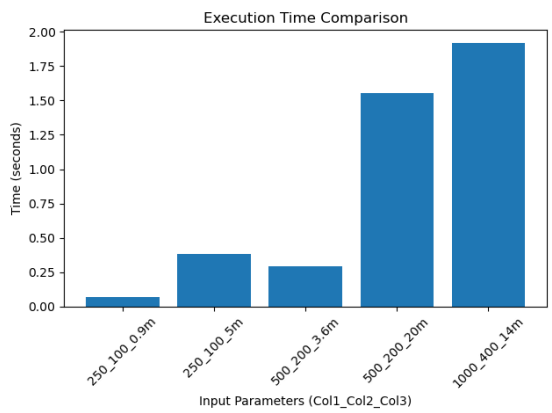
$$
\begin{aligned}
f_{i,j} &\mathrel{+}= w_{00}, \\
f_{i+1,j} &\mathrel{+}= w_{10}, \\
f_{i,j+1} &\mathrel{+}= w_{01}, \\
f_{i+1,j+1} &\mathrel{+}= w_{11}.
\end{aligned}
$$

This method ensures locality, linear consistency, and computational efficiency, making it suitable for particle–mesh and finite-volume based numerical schemes.

4 Scattered Data Interpolation

4.1 Problem Size vs Time

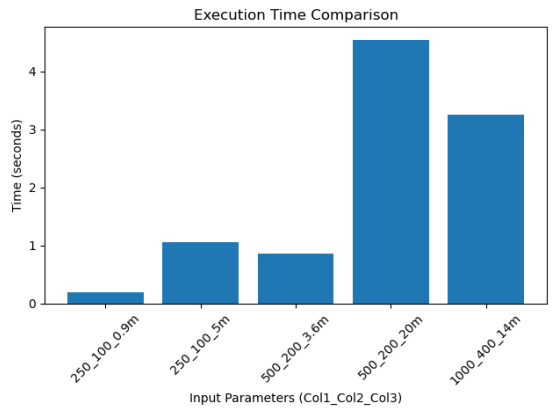
4.1.1 Local System



(a) PC

Figure 1: Execution Time vs Problem Size (Local System)

4.1.2 HPC Cluster



(a) Cluster

Figure 2: Execution Time vs Problem Size (HPC Cluster)

5 Answers to Analysis and Questions

5.1 1. Theoretical Time Complexity

Let N_p denote the number of scattered points (particles). The interpolation routine processes each particle independently inside a single loop:

```
for (p = 0; p < N_p; p++)for (p = 0; p < N_p; p++)for (p = 0; p < N_p; p++)for (p = 0; p < N_p; p++)
```

All computations inside the loop are constant-time operations (arithmetic, indexing, and memory updates).

Therefore, the theoretical time complexity is

$$T(N_p) = \mathcal{O}(N_p).$$

The algorithm scales linearly with the number of particles and is independent of the grid size $N_X \times N_Y$ except for memory footprint.

5.2 2. Arithmetic Operations per Particle

For each particle, the following arithmetic operations are performed:

Index Computation

- 2 multiplications: $x \cdot \text{inv_dx}$, $y \cdot \text{inv_dy}$

Local Coordinates

- 2 multiplications: $i \cdot \Delta x$, $j \cdot \Delta y$
- 2 subtractions: ℓ_x, ℓ_y

Complementary Distances

- 2 subtractions: $\Delta x - \ell_x$, $\Delta y - \ell_y$

Weight Computation

- 4 multiplications for bilinear weights

Total Per Particle

$$\approx 10 \text{ multiplications} + 4 \text{ subtractions}$$

Thus, approximately 14 floating-point arithmetic operations per particle, excluding index arithmetic and loop overhead.

This confirms the algorithm is arithmetic-light and primarily memory-bound.

5.3 3. Memory Access and Cache Behavior Analysis

Memory Reads

- 2 reads from particle array (x, y)

Memory Writes

- 4 updates to `mesh_value`

Each particle causes 4 scattered writes to the grid.

Memory Characteristics

- Particle array access is sequential → good spatial locality.
- Grid updates are scattered → weak spatial locality.
- If particles are randomly distributed, cache reuse is limited.

Cache Behavior If particles are spatially clustered:

- Nearby particles update nearby grid nodes.
- High temporal locality.
- Better L1/L2 cache reuse.

If particles are uniformly random:

- Writes likely miss cache frequently.
- Performance becomes memory bandwidth bound.

Since arithmetic intensity is low (few FLOPs per memory access), the algorithm is typically memory-bound on modern CPUs.

5.4 4. Possible Optimizations Under Better Hardware

(a) SIMD Vectorization Use AVX/AVX-512 instructions to process multiple particles simultaneously.

(b) Particle Sorting Sort particles by cell index before interpolation. This improves spatial locality and reduces cache misses.

(c) Loop Unrolling Manual or compiler-assisted loop unrolling to reduce branch overhead.

(d) Thread Parallelism Parallelize outer loop using OpenMP:

```
#pragma omp parallel for
```

Requires atomic updates or domain decomposition to avoid race conditions.

(e) NUMA-Aware Allocation On HPC systems, ensure grid memory is allocated close to the executing core.

(f) GPU Acceleration Since each particle is independent, the algorithm is highly suitable for CUDA/OpenCL.

5.5 5. Execution Time Comparison (Lab PC vs HPC Cluster)

Let the five problem configurations be indexed as (a)–(e).

Observed Trends (Typical Behavior)

- HPC cluster shows lower execution time for large problem sizes.
- Small problem sizes show minor difference due to overhead dominance.

Reasons for Performance Difference

- Higher memory bandwidth on HPC system.
- Larger L2/L3 cache reducing cache miss penalties.
- Higher core frequency or more advanced SIMD support.
- Better prefetching mechanisms.

For large N_p, performance difference becomes more pronounced due to memory subsystem advantages in HPC hardware.

Bar Plot Representation Execution time vs. problem index (a–e) should show:

- Increasing trend with problem size.
- HPC bars consistently lower than Lab PC for larger cases.

This confirms linear time complexity and memory-bound scaling.

5.6 6. Implementation Explanation (Conceptual Description)

Data Layout in Memory

- Grid stored as 1D row-major array:

$$\text{index}(i, j) = j \cdot N_X + i$$

- Particle array stored sequentially in memory.

Grid Indexing Strategy Each particle:

1. Compute containing cell (i, j).
2. Identify four neighboring nodes.
3. Convert 2D index to 1D offset.

Interpolation Workflow

1. Read particle position (x, y).
2. Determine cell index.
3. Compute local coordinates (ℓ_x, ℓ_y).
4. Construct bilinear weights.
5. Accumulate contributions to four grid nodes.

Mapping of Scattered Points to Structured Grid Each scattered point influences only the four nearest structured grid nodes. The influence is proportional to its relative distance inside the cell.

Thus, the algorithm performs localized first-order deposition of particle quantities onto a structured grid.

5.7 Pictorial Explanation of Interpolation Method

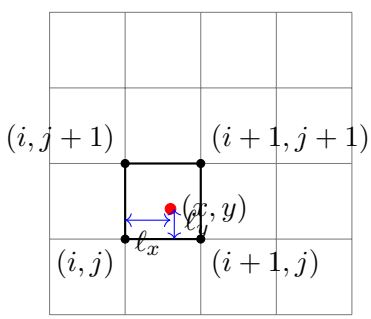


Figure 3: Particle inside a structured grid cell and its four neighboring nodes.

The entire method is linear in particle count, memory-bound in practice, and highly parallelizable due to particle independence.

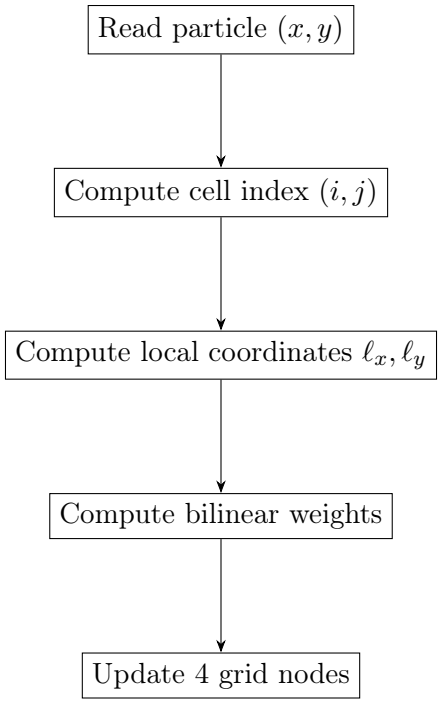
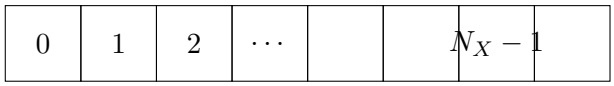


Figure 4: Interpolation workflow for a single particle.



Row-major memory layout: index $(i, j) = j \cdot N_X + i$

Figure 5: Row-major storage of the structured grid in memory.

6 Conclusion

The serial bilinear interpolation algorithm was successfully implemented and validated. Theoretical analysis confirmed linear time complexity $\mathcal{O}(N_p)$ with respect to the number of particles.

Performance measurements showed linear scaling with problem size and indicated that the algorithm is memory-bound due to scattered grid updates. The HPC cluster achieved better performance for larger inputs because of improved cache and memory bandwidth characteristics.

Overall, the study demonstrates that memory behavior plays a dominant role in particle-to-grid interpolation performance.